Final Written Report on

Intelligent Intrusion Detection System

Author:

MCS13

Team members:

Hao Gao 33212333

“Hao Gao wrote the initial drafts and finalised the text for sections 2, 3.”

Jeremy Koh Chi Minn 32744420

“Jeremy wrote the initial drafts and finalised the text for sections 1, 4.1, 5. He prepared materials for Table1, Figure 1 to 6.2, and Sample Code 1.”

Braden Shiew Hao Wen 33273375

“Braden wrote the initial drafts and finalised the text for sections 6, 7.”

Ian Lukose 33926123

“Ian wrote the initial drafts and finalised the text for sections 4.2, 4.3.”

Submission Date: 6/11/2025

Word Count: 3,107

Table of Content

1. Front Cover Sheet 1

2. Introduction 3
    2.1 Project and Project Aims 3
    2.2 Report Structure 3

3. Project Background Summary 5
    3.1 Background Information and Rationale 5
    3.2 Literature Review and Relevant Research 5

4. Outcomes 6
    4.1 Results, Achievements and Delivered product 6
    4.2 Met and Unmet requirements 7
    4.3 Limitations 7

5. Methodology 8
    5.1 Project Rationale 8
    5.2 Project Implementation and Tools 8
    5.3 Sample Code 9

6. Software and Project Critique 11
    6.1 Overview 11
    6.2 Unsuccessful Aspects 11
    6.3 Successful Aspects 11
    6.4 Review 12

7. Conclusion 13

8. References 14

9. Appendices 15
    9.1 Requirement Traceability Matrix 15
    9.2 End Product Screenshots 16
    9.3 Sample Source Code 19

2. Introduction

2.1 Project and Project Aims

Intelligent Network Intrusion Detection System (IDS) is to design and propose a security way which is able to detect threats from the network activities in the real time. As the evolving cyber-attack and the increasing of network usage in today's digital world, cyber-security has become a main concern. The government and each company face an increasing number of sophisticated threats. The traditional intrusion detection system cannot keep up with the modern cyber-attack as they rely on fixed rules and signatures, which makes them less effective.

This project is to overcome these limitations by proposing a proactive way to integrate intelligent methods, such as supervised/unsupervised machine learning, data analysis, to recognize abnormal patterns and predict possible intrusions more accurately. Our main goal is to produce an improved and brand-new version with a more proactive approach and a more accurate model based on the existing IDS. Moreover, our software should be reliable and adaptative that can classify network traffic, detect malicious behavior, and provide real timely alerts to users.

2.2 Report Structure

This report will highlight the project's main goals and objectives, our proposed solution as well as the aims achieved throughout the project in the following structure.

1. Introduction - This part introduces the project and briefly highlight the project background

2. Project background summary - Provide background information to understand the project rationale of how we aimed to design our IDS. This also summarises our literature review to outline related work and relevant research that helped us develop the IDS.

3. Outcomes - Present our project results, detailing what has been achieved in the the development of our IDS and how we implemented machine learning to the monitoring system. This also includes details of our IDS and our AI research.

4. Methodology - Explains the main techniques used to implement the IDS, and the approach we've taken to integrate the machine learning model. This also includes sample code of our IDS.

5. Software and project critique - Highlights how well the project has been executed. Also details what has been done successfully and unsuccessfully from our team's understanding of the project.

6. Conclusion - A summary of our report's key contents.

In conclusion, this report aims to conclude and document the entire development process of our final year project and demonstrate how our team applied the technical knowledge and problem-solving skills to complete this intelligent IDS solution project.

3. Project Background Summary

3.1 Background Information and Rationale

Recently, as more and more people are using the internet for global communication, data exchange and business operations, the internet has become an essential part of model society. However, as the number of devices that connect the internet and the cloud-based services increases, the risk of cyber-attack also increases.

According to Ren et al. (2023), they predict that network data will reach 400 exabytes per month at the end of 2023, which is almost four times more than in 2017. As technology improves, hackers can continue to find various unseen ways to exploit and attack systems which cause heavy financial and security damage worldwide. As mentioned in the Introduction section, Traditional IDS which mainly rely on the fixed-predefined attack signatures, cannot keep up with the model cyber-attacks. It is less effective. Hence, the improvement of IDS in a proactive way (integrating artificial intelligence and machine learning) is necessary. Through existing detection methods and models, anomaly-based detection is most suitable because it identifies unusual network behavior without predefining the abnormal traffic, which is more efficient.

Therefore, the existing anomaly-based IDS is the basic-existing solution, and we integrate the AI to detect the abnormal network traffic to improve the efficiency, accuracy and adaptation of the existing solution.

3.2 Literature Review and Relevant Research

For the literature review, the AI-driven solutions have been explored by many researchers which is in order to improve intrusion detection performance. Studies such as Badr (2022), Zehra et al. (2023), and Liu et al. (2024) illustrate that machine learning and deep learning methods can improve detection accuracy and reduce false positive rate significantly. In addition, different models, including supervised, unsupervised and hybrid models, have been proposed to handle diverse attack types.

For example, reinforcement learning (Badr, 2022) and deep neural networks (Ullah et al., 2025) can identify complex and zero-day attacks more effectively. Other research, such as Ren et al. (2023) and Abu Al-Haija & Zein-Sabatto (2020), emphasize the feature extraction method and data preprocessing. These studies give a foundation for our project, which focuses on building an integration-hybrid version of an intelligent, adaptative anomaly-based IDS.

We expect that it can improve both accuracy and efficiency in detecting modern cyber threats.

4. Outcomes

4.1 Results, Achievements and Delivered product

To achieve our goal of creating a simple IDS for general use, we've designed an IDS that allows users to monitor their network traffic within just a few steps. The simple procedure to start the IDS aims to cover requirement R16 in our traceability matrix. For further details of our requirements, the table (Table 1) shows our Requirement Traceability Matrix.

Our IDS's user interface consists of 4 panels, as shown in (Figure 1).

Start and Stop Feature

The bottom left panel contains an option to choose which network should the IDS monitor, depending on the list of available network options on the specific device. The start button then allows the user to start or stop the network monitoring based on the user's preference, covering requirement (R14).

Graph Feature

At the top left of the screen, a graph is provided to allow users to see how their network is being monitored in real time. The graph presents 2 data points, where the x-axis represents the number of packets being captured, and the y-axis represents the size of each packet being captured, shown in (Figure 2). This covers requirements (R1, R2).

To ensure the IDS is able to perform under high loads of traffic, we've tested the real time packet capture in several traffic situations, as such the IDS is able to handle high loads of traffic data in various situations, this ensures the IDS covers requirements (R10, R11 and R12).

Activity Log Feature

If the basic graph information is insufficient, the user may look to the bottom right, where an activity log is provided, to record, in further detail, each packet being recorded. Each packet recorded shows the data type of the packet, for example TCP, UDP, DATA, etc.; and the length of the packet in integer value. To cover (R13), additional information, such as IP addresses are kept anonymous. An example of the activity log is shown in (Figure 3).

Anomaly Detection Feature

Lastly, the top right of the screen shows a message box, which highlights the anomalies detected during the monitoring session. Each anomaly registered in the message box contains the packet number, and a thread level, based on a base score precalculated based on model training. Any threat score exceeding the base score is flagged as a potential anomaly, where the higher the score, the greater the threat. An example of the message box is shown in (Figure 4). By implementing this feature, the IDS meets requirements (R3, R4, R5, R6)

4.2 Met and Unmet requirements

Based on the Requirement Traceability Matrix, the majority of our system's requirements were successfully met. R1 to R6, which were functional requirements that focused on network monitoring, data collection, alert generation, and intrusion detection were implemented and validated through testing. Similarly, non-functional requirements such as efficiency (R10, R11), availability (R12, R14), security (R13), and usability (R16) were met as well. These outcomes show that the developed IDS operates reliably under test conditions.

However, we had several requirements that remained incomplete or partially achieved. R7, R8 and R9 involved machine learning capability standards, and these requirements were not fulfilled due to time and computational constraints. The model was trained on a dataset of approximately 200,000 packets, while we intended for it to be 20 million packets, which resulted in a model that was not up to our accuracy standards (R7). The inaccuracy of our model also led to our false positives being higher than expected (R15). Adaptive learning (R8) was not implemented as well, as integrating a feedback mechanism to handle flagged false positives proved to be too complex within our project's timeline. Additionally, improving the existing machine learning model (R9) was not achieved as well, as attempts to integrate a supervised model on top of the existing unsupervised model reduced overall accuracy, due to time constraints as well.

4.3 Limitations

Several limitations were identified during the development of our IDS. One major limitation was the computational demand required for training the machine learning model. Due to limited hardware resources, the model was trained on a significantly reduced dataset of approximately 200,000 packets instead of the intended 20 million. This limited our model's exposure to diverse attack patterns and prevented it from achieving the standards we had set for accuracy.

Another limitation involved the system's adaptability to future or unseen cyber threats. As modern attacks evolve rapidly, designing a model capable of adjusting to new forms of malicious behaviour proved highly challenging within the project timeframe. This also affected the implementation of the adaptive learning feature, which was dropped due to the technical complexity and additional processing required for retraining.

Lastly, the attempt to integrate a supervised learning model on top of the existing unsupervised kitNET model led to reduced detection accuracy and instability in predictions. As a result, this hybrid model was not kept towards the end of our project. Overall, the combination of limited computational capabilities, restricted training time, and difficulty of real-time adaptive intrusion detection constrained the project's ability to achieve higher performance metrics.

5. Methodology

5.1 Project Rationale

When designing our project, we aimed to deliver an intrusion detection system that can run in the background, as most security softwares would run real time, to ensure real time security. In addition to that, the software should also be easy to use and straightforward to activate allowing the security feature to be active relatively quickly.

As such, with those aims in mind, we've designed a basic single-menu user interface to include all features for a working IDS. This includes selecting and monitoring a network, and detecting anomalies. (Figure 5) shows our design for this project, with the simplicity in mind.

Single User Interface

The reason for designing a single user interface is to reduce complexity of the IDS. This would prevent clutter and additional background processes, allowing the software to be more compact, and easy to install onto any device.

Use Cases

As our IDS was aimed at any device, be it in a corporate environment or home use, there were several approaches for the functionality and features. This includes making the packet capture information very detailed or very sparse, graphing multiple levels of information or keeping the graph basic. Such things had to be considered, and to make it easier to decide, we've drawn some use cases, to simplify the purpose of our IDS, as shown in (Figure 6.1) and (Figure 6.2).

The conclusion was to keep information to a minimum, only showing basic packet information, like its size, the quantity collected and a threat detection level, as most users who aim to use a security feature would not be able to comprehend the clutter of information of their packet data, let alone own network traffic.

5.2 Project Implementation and Tools

Overview

Our IDS had 2 major components, first was to monitor traffic data in real time, and the other was to analyse the traffic data. The implementation of the software's basis was in reference to Wireshark, a notable free application used to capture packet data.

Pyshark

As such, in order to implement the features provided in wireshark, we aimed to utilise the python library called PyShark. This became the starting point of implementing our IDS. Following this, we've chosen to implement the IDS in python, using several other libraries like Tkinter, threading, matplotlib, and so on.

Tkinter

The use of this library allows us to design basic GUI elements. There are several limitations brought upon our software when using Tkinter as the library only provides basic GUI features, and more modern stylised user interfaces could not be implemented, however as our project aimed to create the basic necessities for an IDS, the visual appeal of the application was not warranted.

Threading

To ensure our application runs smoothly, the user interface and network monitoring run had to run separately. As such, we've used threading to optimise the features, allowing the user interface to run on the main thread, whilst keeping packet capture, and logging information on separate threads. The graph is also updated on a separate thread to ensure proper optimisation for our application.

MatplotLib

Using this library, we were able to implement an efficient graph to plot data points for our packet data monitored during each capture session. The library itself contains more advanced graphing features, but we've limited to using only the simple line graph for easier readability from a user's perspective.

AI Integration (KITSUNE)

For our AI model, we've decided to use an open source library, called KITSUNE. The reason for choosing this model was because it was a python based library, with readily available documentation and a simple feature extractor for packet data.

To integrate the model into our IDS, we first needed to train the model under basic packet data. Which was derived from the CIC-IDS2017 dataset. The method used was a subset of Cross validation. A set of 200,000 packet data was derived from the dataset to train the model; And an additional 4 sets of 50,000 packet data was used to test the trained model. The performance was relatively poor, averaging slightly over 50% accuracy, however this was evident as a more advanced AI model would require several months of training over millions of packet data, and depending on the network traffic provided, the training would also differ.

5.3 Sample Code

As an example of our project, a sample source code shown in (Sample Code 1) describes the method we've used to implement network monitoring in our IDS. The code details a simple child class inheriting a threading class from the threading library, which also includes features derived from the pyshark library.

The sample code shows how the IDS captures packet data on a separate thread, then adds the collected data into a shared data object, which can be accessed by the main thread or other threads to update the data on the user interface.

6. Software and Project Critique

6.1 Overview

After reviewing the entire process, we believe that the project was successful overall as we have managed to deliver an Intelligent Network Intrusion Detection System that achieves our biggest and most important goal – to analyse network traffic in real time and alert users of any detected anomalies.

6.2 Unsuccessful Aspects

Upon deeper inspection, it can be observed that the realised project outcome and initial project proposal are almost exactly one-to-one, with the exception of some requirements we had set for the system's ability to adapt to future threats. The absence of the adaptation requirements stems from a change in how we approached the machine learning aspect of our system. While we were drafting out our requirements initially, we did not have a very deep understanding of the existing models and how they worked. Over the course of our project, we researched and determined that a good choice for our system was KitNET, an unsupervised machine learning model. This model takes in data for training to give each packet a score that is compared with an abnormal threshold that we can calculate after testing is complete to determine if the score is larger than the threshold, making it a threat. This method of determining threats based on a threshold level meant that those requirements could no longer be met due to the nature of the algorithm's simplicity and reliance on manual threshold calculation, making those prior requirements unachievable.

While the missed requirements was one unsuccessful aspect of our project, there was another incident that held us back a little. This incident was a direct result of a slight weakness of our group in the early stages. This weakness was our oversight, where we did not regularly check on our gantt chart, which was made during a free trial. It was only when it was time to submit the gantt chart, did we realise that we had lost access to it. Thankfully, we were able to quickly find a free alternative and remade our gantt chart there. While this incident was not too big of an issue, it taught us to be more vigilant and alert when it comes to ensuring important documents and files are always readily available by regularly checking in on and updating them, rather than completely tossing them aside until needed.

6.3 Successful Aspects

Regarding why we believe that the project was an overall success, the biggest contributors were our teamwork and project management. First, we kept everyone up to date by holding regular meetings for team members and occasionally the supervisor, with the addition of a group chat. These meetings and the group chat ensured that work is always progressing at an appropriate pace and that we can quickly update members and the supervisor on recent developments even if they cannot make it to certain meetings. Next, we effectively delegated tasks based on our individual strengths and weaknesses. By playing to our strengths, we were able to get tasks done efficiently which helped keep the project progressing smoothly. Finally, we went above and beyond in assisting whenever others needed help. For cases

where certain tasks took much longer than others due to their complexities, members who have already completed their parts will always jump in to help with easing the burden of those that are having a rougher time. This combination of effective communication, collaboration and management made it so that we were able to complete our project without any major setbacks.

6.1.4 Review

Overall, while the project has faced some challenges and was not one hundred percent accurate to our initial proposal, we still managed to accomplish our main goal while maintaining a healthy working environment that was harmonious, allowing the process of working on the project to operate at maximum efficiency.

7. Conclusion

To conclude, the Intelligent Network Intrusion Detection System integrates machine learning to monitor and detect anomalies in real time. By using tools such as Tkinter and MatplotLib, combined with the KITSUNE model, the system was successfully completed and is able to capture network packets, visualize traffic, and flag potential threats. Although there were some bumps along the way, such as data loss and failure to meet certain requirements, our effective teamwork and management skills allowed us to persevere and ultimately deliver a functional and intelligent IDS that meets our main goals. Not only has this project strengthened our technical and collaborative skills, but it has also provided us with valuable knowledge regarding cybersecurity solutions involving the use of Artificial Intelligence.

8. References

Abu Al-Haija, Q., & Zein-Sabatto, S. (2020). An Efficient Deep-Learning-Based Detection and Classification System for Cyber-Attacks in IoT Communication Networks. *Electronics (Basel)*, *9*(12), 2152-. https://doi.org/10.3390/electronics9122152

Badr, Y. (2022). Enabling intrusion detection systems with dueling double deep Q-learning. *Digital Transformation and Society*, *1*(1), 115–141. https://doi.org/10.1108/DTS-05-2022-0016

Liu, R., Shi, J., Chen, X., & Lu, C. (2024). Network anomaly detection and security defense technology based on machine learning: A review. *Computers & Electrical Engineering*, *119*, 109581-. https://doi.org/10.1016/j.compeleceng.2024.109581

Ren, K., Zeng, Y., Zhong, Y., Sheng, B., & Zhang, Y. (2023). MAFSIDS: A reinforcement learning-based intrusion detection model for multi-agent feature selection networks. *Journal of Big Data*, *10*(1), 137–30. https://doi.org/10.1186/s40537-023-00814-4

Ullah, B., Massan, S.-R., Rehman, M. A., & Khan, R. A. (2025). Cyber Security Intrusion Detection Using a Deep Learning Method. *Mehran University Research Journal of Engineering and Technology*, *44*(1), 69–74. https://doi.org/10.22581/muet1982.3170

Zehra, S., Faseeha, U., Syed, H. J., Samad, F., Ibrahim, A. O., Abulfaraj, A. W., & Nagmeldin, W. (2023). Machine Learning-Based Anomaly Detection in NFV: A Comprehensive Survey. *Sensors (Basel, Switzerland)*, *23*(11), 5340-. https://doi.org/10.3390/s23115340

9. Appendices

9.1 Requirement Traceability Matrix

REQUIRENENT ID	Description	Categories	Type(s) (FR / NFR)	Source	Status
R1	Continuously monitors network traffic.	Data Visualisation	FR	Dr Fermi	Completed 100%
R2	Keep track of traffic and record flow behaviour	Data Collection	FR	Dr Fermi	Completed 100%
R3	Detecting various types of cyber attack and unauthorized access	Intrusion Detection	FR	Dr Fermi	Completed 100%
R4	Automatic alerts will be shown when an anomaly is detected.	Security alert	FR	Dr Fermi	Completed 100%
R5	Identify anomalies in packet flow behaviour.	Packet Detection	FR	Dr Fermi	Completed 100%
R6	Deduce the current anomaly detected and rate a threat level	Intrusion Detection	FR	Dr Fermi	Completed 100%
R7	Model training: The system will train on historical data and output performance metrics after	Machine Learning	FR	Dr Fermi	INCOMPLETE 0%
R8	Adaptive learning: The system shall allow admins to flag false positives to improve future predictions	Accuracy	FR	Dr Fermi	INCOMPLETE 0%
R9	Produce an improved version of existing machine learning model to predict and detect threats	Machine Learning and intrusion detection	FR	Dr Fermi	INCOMPLETE 0%
R10	The system shall process and analyze network traffic with ≤100ms latency per packet on a single machine	Efficiency	NFR	Dr Fermi	Completed 100%
R11	System should handle ≤10000 packets/second	Efficiency	NFR	Dr Fermi	Completed 100%
R12	System should run for at least 24 hours without crashing	Availability	NFR	Dr Fermi	Completed 100%
R13	System should keep IP addresses anonymous before processing them	Security	NFR	Dr Fermi	Completed 100%
R14	System can be manually turned on or turned off	Availability	NFR	Dr Fermi	Completed 100%
R15	System should maintain ≤15% false positive	Accuracy	NFR	Dr Fermi	INCOMPLET

	rate. Some systems go down to 0.0005%, we can try to achieve 15% with the resources we have				E 0%
R16	Users should be able to set up in ≤10 steps using a guide	Usability	NFR	Dr Fermi	Completed 100%

Table 1: Requirement Traceability Matrix

9.2 End Product Screenshots

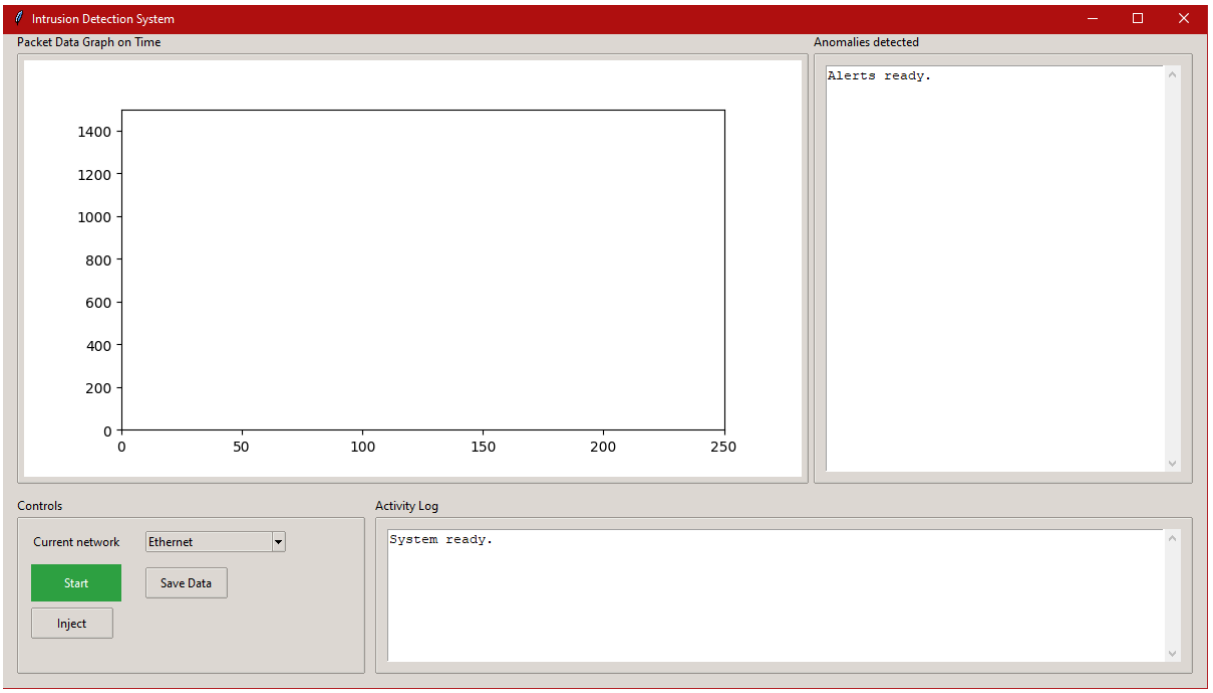


Figure 1: IDS whole user interface

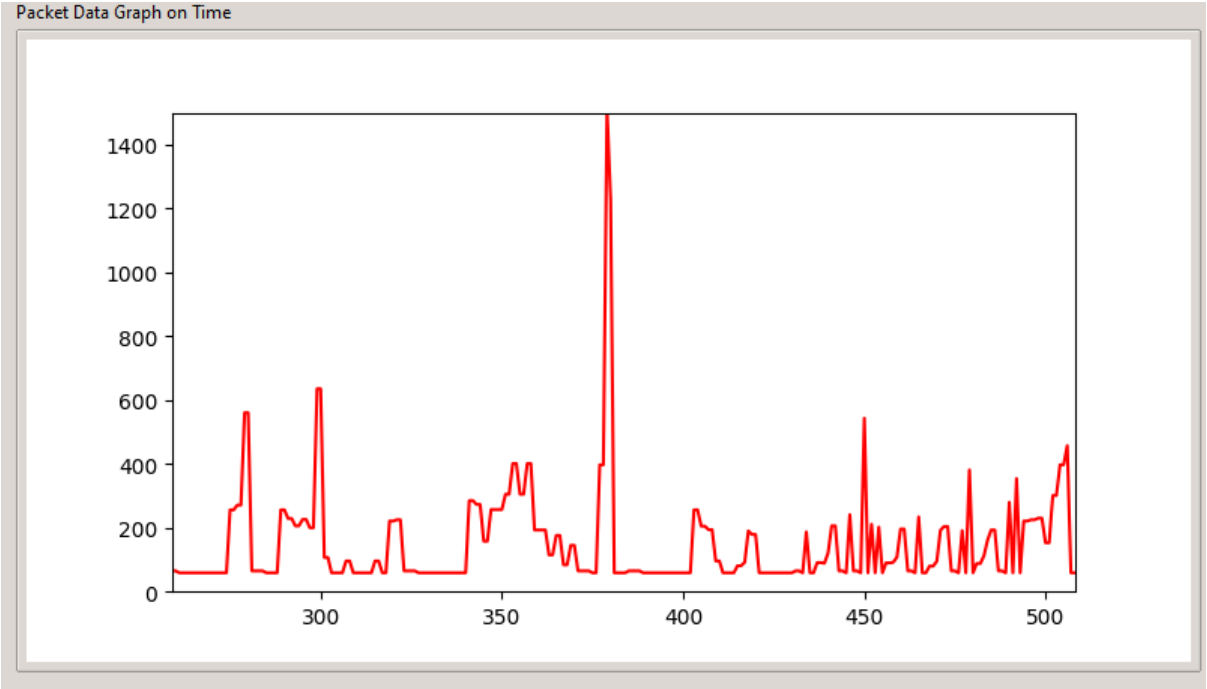


Figure 2: Graph data example

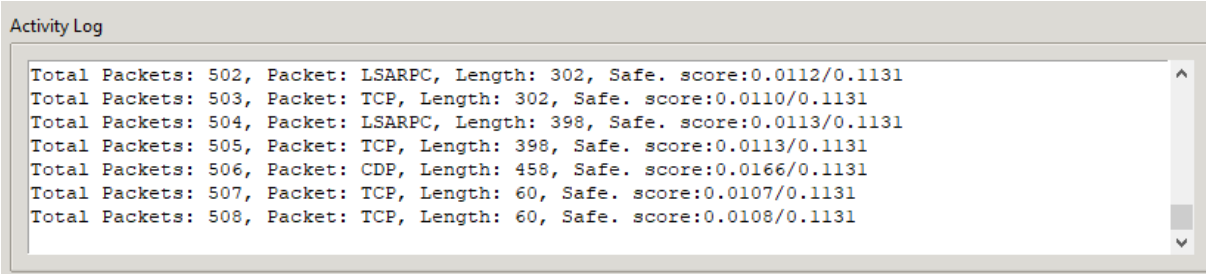


Figure 3: Activity log example

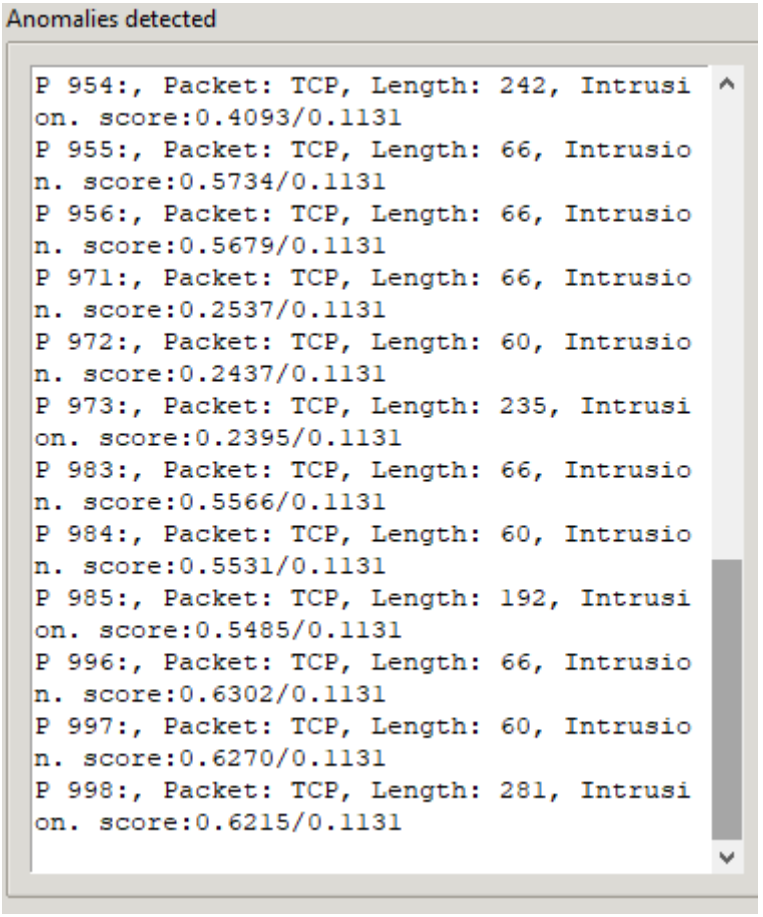


Figure 4: Anomaly detection example

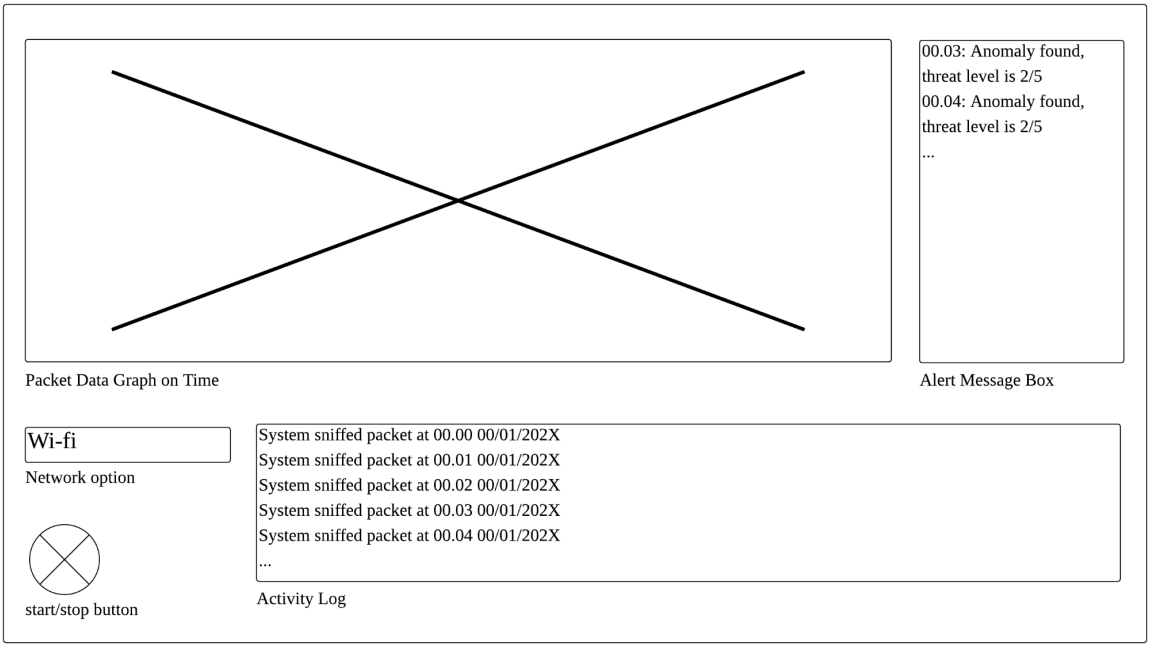


Figure 5: Initial User Interface Design

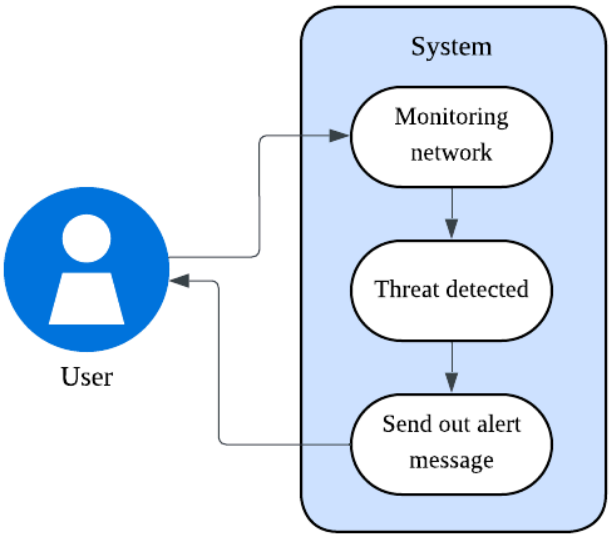


Figure 6.1: Use Case (Threat detection)

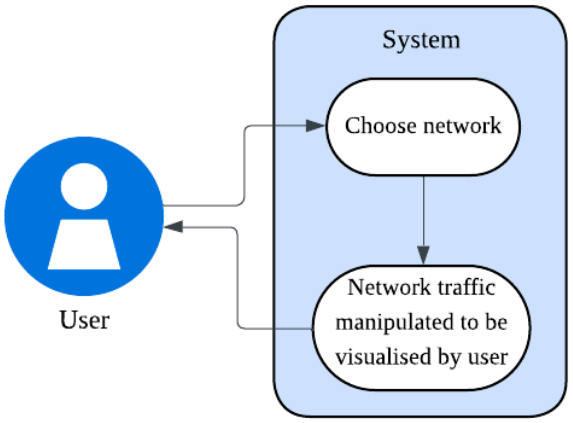


Figure 6.2: Use Case (Monitoring network traffic)

9.3 Sample Source Code

```
import threading
import pyshark
import queue

from test_integration import Model
from networking.nw_shared_data import NWSharedData

"""
This class contains the packet capture thread, used to run packet capture on a separate
thread.
"""
class NWPacketCaptureThread(threading.Thread):
    """
    Initialises the packet capture thread
    args:
        data (NWSharedData): the data shared between threads.
```

```
        capture_stream (LiveCapture) : the live capture stream in the current session.
    """
    def __init__(self, data:NWSharedData, capture_stream:pyshark.LiveCapture = None):
        super().__init__(target=self.capture, daemon=True)
        # add the arguments.
        self._capture_stream = capture_stream
        self._data:NWSharedData = data

        # set the running to true
        self._running = True
        # set the packet count to 0
        self._p_count = 0
        # set the capturing to false
        self._capturing = False

        self.model = Model()

    """
    This function starts the capture loop to obtain packet data.
    """
    def capture(self):
        print("capturing...")
        self._capturing = True
        try:
            for packet in self._capture_stream.sniff_continuously():
                if not self._running:
                    break

                # Getting packet data
                self._p_count += 1

                # Feature extraction for anomaly detection
                score = self.model.detect_intrusion(packet)
                # Add data to the packet queue for graph insertion
                self._data.packet_queue.put([self._p_count, packet.length])

                # Add message to queue to print in logs
                message:str = f"Total Packets: {self._p_count}, Packet:
{packet.highest_layer}, Length: {packet.length}, {score}"
                self._data.packet_message_queue.put(message)

                # Add alert message to the queue to display alert message
                if "Intrusion" in score:
                    self._data.alerts_queue.put(f"P {self._p_count}:, Packet:
{packet.highest_layer}, Length: {packet.length}, {score}")

                print(f"Total Packets: {self._p_count}, Packet: {packet.highest_layer},
Length: {packet.length}, {score}")

        finally:
            # Close the capture stream
            self._capture_stream.close()
            self._p_count = 0
            self._data.reset_queue()
            self._capturing = False
            print("stopping...")
```

```
    """
    This function sets self._running to false.
    """
    def stop(self):
        self._running = False
```

Sample Code 1: Network monitoring Code

Links

CIC-IDS2017
https://www.unb.ca/cic/datasets/ids-2017.html

KITSUNE python library
https://github.com/ymirsky/Kitsune-py

MCS13 Final Written Report Google Document
https://docs.google.com/document/d/1Zl7DTfBOtj8YwpqGN9FZz895iP_s5cb-OOLd9JZUijA/edit?usp=sharing